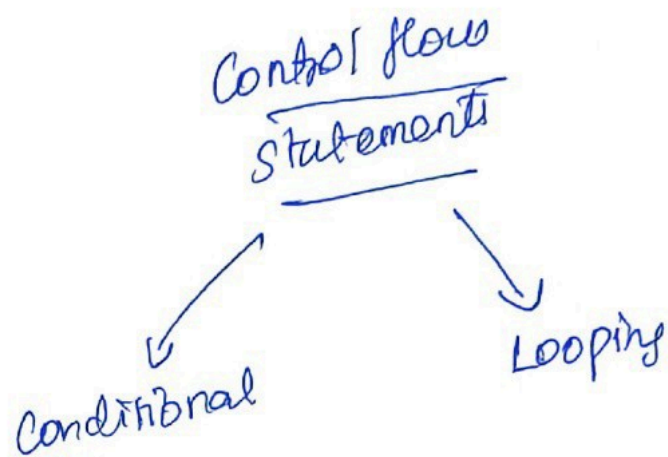




Block scope

{

statements surrounded  
by { }

}

→ Block

scope of variables

{

{

}

→ nested blocks

Java does not allow to ~~redefine~~  
redefine same variable even in nested  
block of that block.

if(condition)

{

}

else

{

}

if(condition)

{

}

else if(Cond.)

{

}

else

{

}

while loop

Executes until  
condition is true  
while (condition)

{

}

do statement  
while condition

do

{

}

while (condition)

do once

for → controlled  
by counter or  
similar variable  
updates iteration

for (initialisation;  
condition; next)

{

}



Be careful of sound off error  
double i;

```
for (i = 0; i <= D-3; i += 20-1)
```

```
{
```

```
}
```

never ends,

==

for each statement

Consider ~~an~~ array

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
|---|---|---|

```
for (int x = 0; x < arr.length)
```

```
{ s = o.p(x);
```

```
}
```

1

2

3

switch

Same thing  
But Different

```
switch ( )
```

```
{ case ->
```

```
case ->
```

```
default ->
```

```
}
```

Case label

→ string literal

→ char, byte  
short int

→ enumerated  
constant

100  
Said  
12/10/2023

# Switch expression

```
int x = switch (season) {  
    case "spring" ->  
        { s-o-pc  
        yield 6 -> give value  
        }  
    default -> -1;  
}
```

Each  
through

```
switch  
runLet = switch ( )  
{  
    case s-o-pc  
    case yield 6  
    default:  
        yield 6;  
}
```

You cannot use return, break  
or continue in switch  
~~statements~~ Expression

# Switch statement

```
switch ( )  
{  
    case -> {  
        }  
}
```

```
switch ( )  
{  
    case:  
    default:  
}
```

break is  
allowed

Break/Continue

Break: stop iteration

Continue: skip iteration

label: do {

do { } while ( )

break label

entire loop break

break label or break  
only next loop  
break

Arrays

Arrays hold sequence of values of  
same type

Declare Arrays

Data structure stores collection  
of values

Access through index  $\rightarrow$  array 1



int[] a

↓  
Declaration

new int[n]; → create memory  
↓ space by JVM  
n = 4

int[] a = new int[n]  
✓

a[0] = 1 → Initialising var

Comma after value is allowed

Initialise with values  
new int[] { 1, 2, 3 };

Legal to have  
Array of len = 0

But arr = new int[] is not same as  
zero length arrays

## Accessing Array Elements

Array elements are accessed by index  
0 to length of array - 1

## Index out of Bounds Exception

If you try access index out of  
the range of elements

Index  $\geq$  length of array  $\rightarrow$  Exception

Index  
0  $\rightarrow$  length of array - 1

length of array  
 $\rightarrow$  array.length  
 $\uparrow$   
✓

## For-each loop

for-each loop is used to iterate  
over elements

```
for (int ele : arr)
```

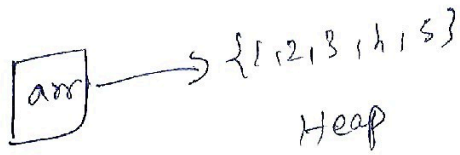
```
{
```

```
}
```

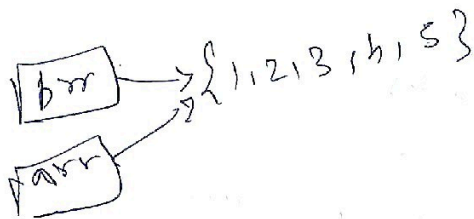
Arrays.toString(a)

$\downarrow$   
convert  
to string

## Copy of Arrays



✓  
~~arr~~ brr = arr



~~Array~~  
Array-copy → also used to declare the length  
- higher than length of array

new int[]  
int[] num = {1, 2, 3};  
✓     ↑     ↓

num = Array-copy (num, 2 \* num.length)

↓

{1, 2, 3, 0, 0, 0}

If length is higher

additional will be

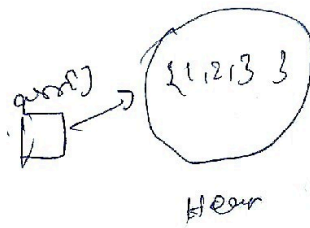
filled with default values

If length is lower

initial values are

copied,





`arr` → predefined  
 Bounds checks

Command-line parameters

```

public class messag
{
    public static void main (String [] args)
    {
    }
}
  
```

java Message -g cool word

`args[0] = -g`  
`args[1] = cool`  
`args[2] = word`

Array sorts //

To sort an Array → Arrays class

java.util Arrays

↓  
`Arrays.sort()`

Sort() is  
tuned version of Quicksort

Algorithm

Arrays fill (arr, a)  $\rightarrow$  fill a to arr

Arrays - binary search

$\downarrow$   
To search operation //

Multidimensional Arrays

It uses more than one index  
to access array elements

$\text{int arr[10][10]} = \text{new int}[10][10];$

for each loop loop through rows

Arrays - deep copy (a)

$\downarrow$   
format multidimensional arrays

|   |   |   |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |